

Lab 06: Filter Responses and Implement PATCH

Maps to: A4, A8

Use response models and exclude_unset semantics for safe partial updates.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Revise the FastAPI product routes so response models exclude internal fields and PATCH uses model_dump(exclude_unset=True). Preserve unspecified fields, define the empty-patch rule and add tests for one-field updates, response filtering, missing products and stable JSON shapes.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Add an internal field to the repository record and confirm it is not in ProductRead.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Declare response_model on list, create and update routes.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Use model_dump(exclude_unset=True) for partial updates.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Patch only stock_quantity and verify SKU, name, price and status remain unchanged.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Send an empty PATCH and apply the documented no-op rule.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Add a regression test for response-field filtering.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

PATCH changes only supplied fields and no internal database field appears in the JSON response.